Course in Advanced Cryptography – June 2025

Course Leader: Michael Scott (Technology Innovation Institute)

This course will concentrate on Implementation/Application of Elliptic Curve Cryptography and Cryptographic pairings, much less on Mathematics.

The main components will be:-

- Construction of a Crypto Library in Python. Review of cyclic groups and basic number theory.
- Elliptic Curve Cryptography. Point Counting.
- Pairings, Type 1 and Type 3. Discrete Logarithm Algorithms. Group Structure
- Efficient Implementation of Pairings. Pairing-Friendly Curves.
- Pairing-based Protocols. Identity-Based Encryption, Short Signatures, Identification algorithms, attribute-based crypto.

During the course, you will develop your own crypto library in Python. Almost all methods described will be implemented.

Simple Modular Arithmetic in Python

Create this simple Python program in a file *ex1.py*

```
a=int(input("a= "))
b=int(input("b= "))
c=a+b
print("c= ",c)
```

To compile and run the program

```
py ex1.py
```

or maybe

```
python3 ex1.py
```

In Cryptography we use really big numbers! Fortunately Python supports such numbers. Enter some really big numbers and run the program again.

Next let's try and do arithmetic modulo a prime p=11.

```
p=11
a=int(input("a= "))
b=int(input("b= "))
c=(a+b)%p
print("c= ",c)
```

Calculating (*a*-*b*) mod *p* is a bit more tricky

```
p=11
a=int(input("a= "))
b=int(input("b= "))
c=(a-b)%p
if c<0 :
    c+=p
print("c= ",c)
```

However calculating *a.b* mod *p* is easy.

```
p=11
a=int(input("a= "))
b=int(input("b= "))
c=(a*b)%p
print("c= ",c)
```

These are such important operations, we will put them into a module, called *numbers.py*.

```
def modadd(a,b,p) :
    c=(a+b)%p
    return c

def modsub(a,b,p) :
    c=(a-b)%p
    if c<0 :
        c+=p
    return c

def modmul(a,b,p) :
    c=(a*b)%p
    return c
```

Calculating *a*/*b* mod *p* is more complex – we will come back to that.

Next we will calculate the values $y=2^x$ mod *p* and print them out for positive *x*. Store the following in *exp.py*

```
import numbers

p=11
g=2
y=1
for x in range(1,24) :
    y=numbers.modmul(y,g,p)
    print("y= ",y)
```

Compile and run the program

```
py exp.py
```

We are generating elements in a cyclic group. Observe that 2^{10} mod 11 = 1 (Fermat's Theorem). Now try it again using 3 as a *generator* instead of 2.

Since 2 generates all of the elements of the group from 1 to p-1, we call 2 a *primitive root* of 11. We say that 3 is a generator of a prime-order subgroup, of order 5, as it only generates 5 elements before cycling, and 5 is prime.

We also say

$Ord_{11}(2) = 10$

And

$Ord_{11}(3) = 5$

Note that the Order of an element *must* be a divisor of p-1, as a consequence of Fermat's theorem.

Also a generator g is a primitive root if and only if $g^{(p-1)/q} \neq 1$ for any exact divisor of p-1.

Next we introduce the *Euler Totient Function*, denoted $\varphi(n)$. This is the number of numbers less than n which are relatively prime to n. Clearly $\varphi(p)=p-1$.

What is $\varphi(10)$? The numbers less than 10 with no factors in common with 10 are {1,3,7,9}. So $\varphi(10)=4$.

Euler's theorem - $g^{\varphi(n)} \bmod n = 1$ for any n, if g and n are co-prime. For example $3^4 = 81 = 1 \bmod 10$.

Now try the program above, this time with n=10 and g=3.

The number of primitive roots mod a prime p is $\varphi(p-1)$. So there are 4 primitive roots mod 11.

Euler's theorem allows us to conclude that

$$g^x \bmod n = (g \bmod n)^{x \bmod \varphi(n)} \bmod n$$

So – What is the least significant decimal digit of 103^{441}? Its $103^{441} \bmod 10 = 3^1 \bmod 10 = 3$.

Homework: Use the computer to find all the primitive roots mod 11

Homework: What is the least significant decimal digit of 100007^{876567} ?

Jacobi Symbol

Observe that $3^5 \bmod 11 = +1$ and $2^5 \bmod 11 = -1$ (10 mod 11 can be considered as -1 mod 11).

Clearly $g^5 \bmod 11$ is just the square root of $g^{10} \bmod 11=1$, and so it must be -1 or +1.

The value of $g^{(p-1)/2} \bmod p$ is so important it is given its own symbol (g/p), the Legendre symbol, which can only have the values 0,+1 or -1.

Jacobi extended the idea to (g/n) where n is odd. Note that if $n=p.q$ then $(g/n) = (g/p).(g/q)$

Euler proved

$(0/n) = 0$
$(1/n) = 1$
$(g/n) = (g \bmod n/n)$
$(2/n) = (-1)^{(n.n-1)/8}$
$(ab/n) = (a/n).(b/n)$
$(x/pq) = (x/p).(x/q)$
$(g/n) = (-1)^{(n-1)(g-1)/4}.(n/g)$

This last is called the *law of quadratic reciprocity*.

These rules can be used to efficiently evaluate the Jacobi symbol *without* knowing the factorisation of n.

Quadratic Residues

Now consider the effect of squaring each element in a group mod p=11

```
import numbers

p=11
g=2
y=1
for x in range(1,p) :
    y=numbers.modmul(x,x,p)
    print(x,"^2 mod ",p," = ",y)
```

There is an obvious symmetry as $(-x)^2 = x^2 \bmod p$

The elements which are generated by this process are called *quadratic residues* (QRs). The elements which are not generated are called the *quadratic non-residues* (QNRs). So in this case {1,4,9,5,3} are the quadratic residues, and {2,6,7,8,10} are the quadratic non-residues.

Clearly a QR in the case of a prime modulus has two square roots. For example 5 has 4 and 7 as square roots mod 11 (Check: 4.4=16=5 mod 11, 7.7=49=5 mod 11). In fact 4 and 7 can be regarded as +4 and -4 mod 11.

So the square roots of 5 mod 11 are ±4.

However a QNR has no square roots at all!

How can you tell if x is a QR modulo a prime p?

It is *if and only if* (x/p)=+1

Homework: Find the primitive roots and quadratic residues mod 13

Now add a function to `numbers.py` to calculate the Jacobi Symbol.

```
def jacobi(a,p) :
    m=0;
    if a<1 or p<2 or (p%2 == 0) :
        return 0
    x=a%p

    while p>1 :
        if x == 0 :
            return 0
        n8=p%8
        k=0;
        while x%2 == 0 :
            k=k+1
            x/=2
        if k%2 == 1 :
            m+=(n8*n8-1)/8
        m+=(n8-1)*((x%4)-1)/4
        t=p
        t%=x
        p=x
        x=t
        m%=2
    if m == 0 :
        return 1
    else :
        return -1
```

Now create a test program *jack.py*

```
import numbers

x=int(input("x= "))
n=int(input("n= "))

j=numbers.jacobi(x,n)

if j==+1 :
    print("x/n=+1")
if j==-1 :
    print("x/n=-1")
```

Run the program

If $n=p.q$ where p and q are both primes, then $(x/n)=(x/p).(x/q)$. But x is only a quadratic residue mod n, if $(x/p)=(x/q)= +1$

Use the program to test this relationship

Homework: Is 123456 a quadratic residue mod the prime 268435399?
Homework: 974153 is the product of two primes. Using the program above, can you determine if 11 is a QR? Is 13 a QR?